Managing Secrets

When we defined our seed data back in *Iteration A2: Making Prettier Listings*, our database didn't have any users. After we added our first user in Chapter 14, *Task I: Logging In*, we added code to require a valid login to access pages that update the database.

What that means is that if we were to deploy a new installation starting with the seed data alone, we would be locked out of our own application. That's not good. So lets fix it!

Adding an initial user to our seed data solves the problem, but checking in a password into our version control and deploying it is hardly secure. Fortunately Rails has provided a way to encrypt secrets such as this one. Rails calls such secrets credentials.

We get started by editing our credentials:

```
$ EDITOR='code --wait' rails credentials:edit
```

Feel free to replace the editor with `vim` or another editor of your choice.

You'll see that Rails has already defined one credential that's used to encrypt cookies, which is how Rails implements sessions such as the one used to track a user's cart. Leave that credential alone, and add another one to the file:

```
dave_password: secret
```

When you save the file, Rails will update `config/credentials.yml.enc` using the master key defined in `config/master.key`. The encoded file can be checked into version control and shared publicly. The key, however, needs to be kept private.

If you look into the `.gitignore` file, you'll see that `/config/master.key` is already listed there. For similar reasons, we won't want the key to be placed into the docker image that we'll be creating shortly, so we'll want to create a `.dockerignore` file with this in it. As the `.gitignore` is a good starting point, we can simply copy